React + TypeScript · 10-Day Training Plan

Day 2 — TypeScript

Your simple, friendly guide for today

COMING FROM	KnockoutJS / .NET (C#) → moving to React + TypeScript
TODAY'S GOAL	Add C#-style types on top of the JavaScript from Day 1, so mistakes get caught while you type — not when the app crashes.

How to use this: First read **Section 1 (Learning Plan)** to see each topic with a small example. Then read **Section 2 (Detailed Notes)** slowly, top to bottom — one read should make everything click. Keep **Section 3 (Common Mistakes)** handy while you practise.

1 Learning Plan

Day 2 — TypeScript (Learning Plan, first 2 hours)

Welcome to Day 2. Yesterday you learned modern JavaScript. Today we add **types** on top of it. That word "types" should feel warm and familiar — you have used types in C# every single day. TypeScript is basically **JavaScript + a type system that looks a lot like C#'s**. Same idea: catch mistakes while you type, not when the app crashes.

The plan is simple: read each topic, type the tiny snippet yourself, and watch how the editor underlines your mistakes in red *before* you even run anything. That red underline is your new best friend.

One rule for today: everything here compiles under **strict mode** (the safest, most helpful setting). Where I want to *show* you a mistake, it lives in a comment starting with `// ❌ Error:` — never as real code.

Part A — The Official Topics

1. Primitive Types

Primitive types are the basic building-block types: `string`, `number`, `boolean`, plus a few TypeScript-specific ones. You write the type after a colon, like `name: string`. Once a variable has a type, TypeScript stops you from putting the wrong kind of value in it.

```
let firstName: string = "Het";
let salary: number = 50000;
let isActive: boolean = true;

// TypeScript can also GUESS the type for you (called "inference"):
let department = "Engineering"; // TypeScript knows this is a string
// ❌ Error: Type 'number' is not assignable to type 'string'
// department = 5;
```

- **.NET / C# contrast:** Almost identical to `string name`, `int salary`, `bool isActive` — the type just moves *after* the name with a colon (`name: string`) instead of before it.
- **You'll be able to...** label your variables with types and let the compiler catch wrong-value mistakes as you type.

---

2. Interfaces

An interface describes the **shape** of an object — what properties it has and their types. It is the perfect way to give your Day-1 Employee object a proper type. It only describes shape; it produces no real code at runtime.

```
interface Employee {
  id: number;
  firstName: string;
  lastName: string;
  department: string;
  salary: number;
  isActive: boolean;
  joinDate: string;
}

const emp: Employee = {
  id: 1,
  firstName: "Het",
  lastName: "Shah",
  department: "Engineering",
  salary: 50000,
  isActive: true,
  joinDate: "2024-01-15",
};
```

- **.NET / C# contrast:** Just like a C# `interface` (or a simple data class / DTO) — a contract for what an object must contain.
- **You'll be able to...** give your Day-1 Employee a real type, so a missing or wrongly-typed field is flagged instantly.

---

3. Type Aliases (`type`)

A `type` alias gives a name to any type. It can do everything an interface does for object shapes, *plus* it can name things interfaces can't — like a **union** (this OR that). Unions are hugely useful for "one of a fixed set of values."

```
// A union type: the value must be one of these exact strings
type Department = "Engineering" | "Sales" | "HR" | "Finance";

// A type alias for an object shape (like an interface)
type EmployeeName = {
  firstName: string;
  lastName: string;
};

let dept: Department = "Sales";
// ❌ Error: Type '"Marketing"' is not assignable to type 'Department'
// dept = "Marketing";
```

- **.NET / C# contrast:** The union `"Engineering" | "Sales" | ...` is like a safer, lighter `enum` — but C# has no direct equivalent to string unions, so this is a genuinely new (and very handy) tool.
- **You'll be able to...** name reusable types and restrict a value to a fixed set of allowed options.

Quick rule of thumb: use `interface` for object shapes (especially ones others might extend), and `type` when you need unions or want to name something that isn't a plain object. In practice they overlap a lot — don't overthink it.

---

4. Enums

An enum is a named set of related constant values. It groups options under one name, so you write `Department.Engineering` instead of loose strings. TypeScript enums are one of the few features that *do* create real code at runtime.

```
enum Status {
  Active,   // = 0
  OnLeave,  // = 1
  Resigned, // = 2
}

let empStatus: Status = Status.Active;

// You can also fix the values as strings (often clearer in logs/APIs):
enum StringStatus {
  Active = "ACTIVE",
  OnLeave = "ON_LEAVE",
  Resigned = "RESIGNED",
}
```

- **.NET / C# contrast:** This is basically C#'s `enum` — same concept, same auto-numbering from 0, and you can assign explicit values just like in C#.
- **You'll be able to...** replace loose "magic strings" with a clean named set of statuses.

> Tip: many TypeScript teams prefer a **string union** (`type Status = "Active" | "OnLeave" | "Resigned"`) over `enum` because it's simpler and leaves no runtime code. Know both; you'll see both.

---

5. Generics

Generics let you write a type or function that works with **any type you plug in later**, while staying fully type-safe. You'll use this constantly for API responses — the response wrapper is the same, but the `data` inside changes (Employee, Department, etc.). The `<T>` is a placeholder you fill in.

```
// A reusable API response wrapper — reuse from Day 1's mock API idea
interface ApiResponse<T> {
  success: boolean;
  data: T;
  error?: string;
}

// Now reuse it with different data types:
const empResponse: ApiResponse<Employee> = {
  success: true,
  data: emp, // must be an Employee
};

const listResponse: ApiResponse<Employee[]> = {
  success: true,
  data: [emp], // must be an array of Employees
};
```

- **.NET / C# contrast:** Identical in spirit to C# generics — `ApiResponse<T>` is just like `ApiResponse<T>` or `List<T>`, and `<T>` is your type parameter.
- **You'll be able to...** write one `ApiResponse<T>` and reuse it for every entity, with full type checking on `data`.

---

6. Utility Types (`Partial` / `Pick` / `Omit` / `Required` / `Readonly` / `Record`)

Utility types are built-in helpers that build a *new* type from an existing one, so you don't repeat yourself. For example, an "update" form usually allows *some* fields, so `Partial<Employee>` makes every field

optional. These map beautifully onto your Day-1 CRUD service.

```
// Partial<T>: all fields optional — perfect for an "update" payload
type EmployeeUpdate = Partial<Employee>;
const patch: EmployeeUpdate = { salary: 60000 }; // only the fields you change

// Pick<T, Keys>: keep only some fields
type EmployeeSummary = Pick<Employee, "id" | "firstName" | "department">;

// Omit<T, Keys>: everything EXCEPT some fields (great for "new, no id yet")
type NewEmployee = Omit<Employee, "id">;

// Required<T>: make every field mandatory (opposite of Partial)
type FullPatch = Required<EmployeeUpdate>;

// Readonly<T>: no field can be reassigned after creation
type FrozenEmployee = Readonly<Employee>;

// Record<Keys, Value>: build an object type from keys → value
type CountByDepartment = Record<Department, number>;
const counts: CountByDepartment = {
  Engineering: 12,
  Sales: 8,
  HR: 3,
  Finance: 5,
};
```

- **.NET / C# contrast:** C# has no built-in equivalent — you'd hand-write separate DTOs (a `CreateEmployeeDto`, an `UpdateEmployeeDto`, etc.). Utility types generate those variations for you automatically.
- **You'll be able to…** derive create/update/summary/read-only versions of `Employee` without copy-pasting the fields.

---

7. Optional Properties

An optional property is one that *might* not be there. You mark it with a `?` after the name (`middleName?: string`). TypeScript then makes you check for `undefined` before using it, which prevents null-reference-style bugs.

```
interface EmployeeProfile {
  id: number;
  firstName: string;
  middleName?: string; // optional — may be undefined
}

const p: EmployeeProfile = { id: 1, firstName: "Het" }; // fine, no middleName

// You must handle the "might be missing" case:
const initial = p.middleName?.charAt(0) ?? "-"; // safe access + fallback
```

- **.NET / C# contrast:** The `?` marker is like C#'s nullable types (`string?`), and `?.` / `??` are the exact same optional-chaining and null-coalescing operators you know from C#.
- **You'll be able to...** model fields that may be absent and let the compiler force you to handle the missing case.

---

8. Type Narrowing (`typeof`, the `in` operator, discriminated unions)

Narrowing means TypeScript figures out a *more specific* type inside an `if` block, based on a check you wrote. After you check something, TypeScript "narrows" the type and lets you use the matching properties safely. Three common tools: `typeof` (for primitives), `in` (does this property exist?), and **discriminated unions** (a shared "tag" field that tells the variants apart).

```
// (a) typeof — narrow a primitive union
function formatId(id: string | number): string {
  if (typeof id === "string") {
    return id.toUpperCase(); // here TS knows id is a string
  }
  return id.toFixed(0); // here TS knows id is a number
}

// (b) 'in' — narrow by checking a property exists
interface Manager { teamSize: number; }
interface Intern { mentor: string; }
function describe(person: Manager | Intern): string {
  if ("teamSize" in person) {
    return `Manages ${person.teamSize}`; // TS knows it's a Manager
  }
  return `Mentored by ${person.mentor}`; // TS knows it's an Intern
}

// (c) Discriminated union — a shared "kind" tag picks the variant
type LoadState =
  | { kind: "loading" }
  | { kind: "success"; data: Employee[] }
  | { kind: "error"; message: string };

function render(state: LoadState): string {
  switch (state.kind) {
    case "loading": return "Loading…";
    case "success": return `Loaded ${state.data.length}`; // data is safe here
    case "error":   return state.message;                 // message is safe here
  }
}
```

- **.NET / C# contrast:** Discriminated unions feel like C# pattern matching with a `switch` on a type/tag — the compiler even checks you've handled every case. `typeof` is like C#'s `is` type check.
- **You'll be able to…** safely handle "this could be several shapes" values, which is exactly how you'll model loading/success/error states in React.

---

9. tsconfig

`tsconfig.json` is the settings file for the TypeScript compiler. It tells TS which files to compile, which JavaScript version to output, how strict to be, and where to put results. Every TypeScript project has one,

and you'll create it with one command (see Part B).

```
{
  "compilerOptions": {
    "strict": true,
    "target": "ES2020",
    "module": "ESNext"
  }
}
```

- **.NET / C# contrast:** Think of it as your `.csproj` / build settings for TypeScript — the knobs that control how source turns into runnable output.
- **You'll be able to…** configure a project so the compiler enforces the strict, safe rules we use all week.

Part B — Extra Topics: Setup, Tools & Mindset

Everything you need to actually get TypeScript running today. Short and practical.

B1. Setting up TypeScript in a project

Install TypeScript into your project (the `-D` means "dev dependency" — only needed while developing, not at runtime), then generate a config file:

```
npm i -D typescript
npx tsc --init
```

`npx tsc --init` creates a `tsconfig.json` full of commented options. You'll trim it down to the essentials below.

- **Coming from .NET:** `npm i -D typescript` is like adding a NuGet dev tooling package; `tsc` is the TypeScript compiler, the rough equivalent of `csc` / `dotnet build` for C#.

B2. A minimal, sensible `tsconfig.json`

Here is a clean starting config with plain-English notes:

```
{
  "compilerOptions": {
    "target": "ES2020",          // JS version to output (modern, widely supported)
    "module": "ESNext",          // use modern import/export from Day 1
    "moduleResolution": "Bundler",// how TS finds imported files (good modern default)
    "strict": true,              // turn ON all the safety checks — keep this true
    "esModuleInterop": true,     // smoother imports of older CommonJS packages
    "skipLibCheck": true,        // don't re-check library type files (faster builds)
    "forceConsistentCasingInFileNames": true, // avoid Windows/Linux path casing bugs
    "outDir": "dist",            // put compiled .js files here
    "rootDir": "src"             // your source .ts files live here
  },
  "include": ["src"]             // only compile files in the src folder
}
```

The single most important line is `"strict": true`. It switches on all the good checks (like catching possible `undefined`), and it's why our code today is careful about optional fields.

B3. How to actually RUN TypeScript day-to-day

Two everyday options:

```
# Option 1: run a .ts file directly (fastest inner loop) — install tsx once:
npm i -D tsx
npx tsx src/index.ts

# Option 2: compile to JavaScript, then run with Node:
npx tsc            # produces .js files in dist/
node dist/index.js
```

For learning and quick experiments, `tsx` (or the older `ts-node`) is the smoothest — you just run the `.ts` file, no separate build step. Use `tsc` when you want to produce real `.js` output.

- **Coming from .NET:** `npx tsx file.ts` feels like `dotnet run` (run straight from source); `tsc` then `node` feels like `dotnet build` then running the produced binary.

B4. VS Code TypeScript tips

VS Code has TypeScript built in — no extension needed for type checking.

- **Red squiggles = type errors.** They appear as you type, before running anything. Hover to read the full message.
- **Hover any variable** to see its inferred type — great for learning what TS figured out on its own.

- **Quick Fix (`Ctrl+.`)** offers auto-fixes: add a missing import, fill in missing interface properties, change a type, etc.
- **F12 (Go to Definition)** jumps to where a type or function is defined — just like in Visual Studio.

B5. ESLint for TypeScript (`typescript-eslint`)

Why: the TypeScript compiler checks *types*; ESLint checks *code quality and patterns* (unused variables, risky `any`, missing `await`). Different jobs — use both, exactly like Day 1.

Minimal setup:

```
npm i -D eslint typescript-eslint
```

Then a tiny flat config file `eslint.config.js`:

```
import tseslint from "typescript-eslint";

export default tseslint.config(...tseslint.configs.recommended);
```

Run it with `npx eslint .`. That's enough to start catching TypeScript-specific smells.

B6. Coming from C#/.NET — what's familiar, what's different

Feels familiar (you already know this):

- Static types on variables, parameters, and return values.
- `interface` as a contract for shape.
- Generics with `<T>` (just like `List<T>`).
- `enum` for named constant sets.

Genuinely different — remember these two:

- **Structural typing ("duck typing").** In C#, a class matches an interface only if it explicitly says `: IEmployee`. In TypeScript, any object matches a type **if it has the right shape** — no `implements` needed. If it has all the required fields with the right types, it fits.
- **Types are erased at runtime.** TypeScript types vanish once compiled to JavaScript — there is no reflection on them, no `typeof MyInterface`. So you can't do a runtime check "is this really an Employee?" the way C# reflection allows. (Enums are the main exception — they leave real code behind.)

```
// Structural typing in action — no "implements" keyword needed:
interface HasName { firstName: string; }

function greet(x: HasName) {
  return `Hi ${x.firstName}`;
}

// This works because emp HAS a firstName — even though it was typed as Employee:
greet(emp); // ✅ fits the shape, so it's allowed
```

Quick recap

By the end of these two hours you should be able to type your Day-1 Employee module: an `Employee` interface, a `Department` union, a generic `ApiResponse<T>`, `Partial<Employee>` for updates, optional fields with `?`, and safe narrowing for loading/success/error states — all under `strict` mode, run with `tsx` or `tsc`. Tomorrow (Day 3) we start putting these typed shapes into real React components.

2 Detailed Notes — Read Once, Understand Everything

The big idea first

TypeScript is just JavaScript with types bolted on top. Think of it as a **thin typed layer** sitting over the JavaScript you learned yesterday. You write almost the same code, but you also get to say *"this thing is a* `string` *, that thing is a* `number` *"* — and TypeScript checks those promises **before you ever run the code**. If you try to put a number where a string belongs, it complains right there in your editor, red squiggle and all.

If you're coming from C#, you already know this feeling. C# checks your types at compile time; TypeScript does the same for JavaScript. The type system here is genuinely close to C#'s — interfaces, generics, enums, they all rhyme with what you know.

One crucial thing to burn into your brain early: **the types disappear when the code runs.** TypeScript compiles down to plain JavaScript, and every type annotation is stripped out. The types are like scaffolding on a building — they help you build it correctly, then they're gone before anyone moves in. This is different from C#, where types are still there at runtime (reflection, `typeof`, etc.). In TypeScript, at runtime it's *just JavaScript*. That's why later we'll need runtime checks like `typeof` to tell types apart — because the type labels aren't around anymore to ask.

Okay, let's walk through it all.

---

1. Primitive types and type annotations

A **primitive** is a basic single value — text, a number, true/false. You add a type by writing a colon and the type after the variable name. That colon-and-type is called a **type annotation** (a little label saying what kind of value is allowed).

```
// primitive types with annotations (the ": type" part)
let firstName: string = "Ada";
let salary: number = 85000;
let isActive: boolean = true;
let joinDate: Date = new Date();

// null and undefined are their own types too
let middleName: string | null = null;
```

C# analogy: `string firstName = "Ada";` in C# becomes `let firstName: string = "Ada";` here. The type just moves to *after* the name. Note there's only one `number` (no separate `int` / `double` / `decimal`) and one `string`. Simpler than C# in that respect.

---

2. Type inference — let TypeScript do the work

You often **don't** need to write the type at all. If you give a variable a value right away, TypeScript figures out the type for you. This is called **type inference** (TypeScript *inferring*, i.e. guessing correctly, the type from the value).

```
// TypeScript infers the type from the value — no annotation needed
let department = "Engineering"; // inferred as string
let headcount = 12;             // inferred as number

// ❌ Error: Type 'number' is not assignable to type 'string'.
// department = 5;
```

This is like C#'s `var` — `var department = "Engineering";` and the compiler knows it's a string. Rule of thumb: **don't over-annotate.** If the value makes the type obvious, leave the annotation off. Annotate function parameters and public shapes; let inference handle simple local variables.

---

3. Interfaces vs type aliases

Both let you name a shape. Here's the plain difference:

- `interface` — describes the shape of an **object**. Very much like a C# interface or a simple data class shape.
- `type` (a **type alias** — just a nickname for a type) — can name **any** type: an object, a union, a primitive, whatever.

```
// interface: describes the shape of an object
interface Employee {
  id: number;
  firstName: string;
  lastName: string;
  department: string;
  salary: number;
  isActive: boolean;
}

// type alias: gives a name to ANY type, not just object shapes
type ID = number;
type EmployeeName = { firstName: string; lastName: string };

const emp: Employee = {
  id: 1,
  firstName: "Ada",
  lastName: "Lovelace",
  department: "Engineering",
  salary: 85000,
  isActive: true,
};
```

Simple rule for when to use which: use `interface` for object shapes (especially things others might extend, like props objects in React). Use `type` when you need something an interface can't do — a union, a primitive alias, or combining types. When both would work for an object, either is fine; pick `interface` and move on.

---

4. Optional properties (`?`) and readonly properties

Two small markers you'll use constantly:

- `?` after a property name means **optional** — it's allowed to be missing.
- `readonly` means you can set it once, then it's frozen — no reassigning.

```
interface Employee {
  readonly id: number;      // can set once, never change (like C# readonly / init)
  firstName: string;
  middleName?: string;      // optional — may be missing (undefined)
  lastName: string;
}

const emp: Employee = { id: 1, firstName: "Ada", lastName: "Lovelace" };
// middleName is fine to leave out

// ❌ Error: Cannot assign to 'id' because it is a read-only property.
// emp.id = 2;
```

C# analogy: `readonly` here maps neatly to C#'s `readonly` field or an `init`-only property. An optional `middleName?: string` is basically saying its value might be there or might be `undefined`.

---

5. Union types and literal types

A **union type** means "this can be one of several types." You write it with a `|` (pipe) between the options — read it as the word "or".

A **literal type** is even more precise: the value must be one *exact* value, not just any string. Combine a few literals with `|` and you get a tidy little set of allowed values.

```
// literal type: the value can ONLY be one of these exact strings
type Status = "active" | "inactive" | "onLeave";

// union type: a value that can be one of several types
type IdOrCode = number | string;

let status: Status = "active";
status = "onLeave"; // fine

// ❌ Error: Type '"retired"' is not assignable to type 'Status'.
// status = "retired";

let key: IdOrCode = 42;
key = "EMP-42"; // also fine
```

This literal-union trick is one of TypeScript's most-loved features and C# doesn't have a direct equivalent. `Status` behaves a bit like an enum, but it's just plain strings under the hood — no extra machinery.

---

6. Enums (and why a string union is often simpler)

An **enum** is a named set of constants — exactly like a C# `enum`.

```
// enum: a named set of constants (just like a C# enum)
enum Department {
  Engineering = "Engineering",
  Sales = "Sales",
  HR = "HR",
}

let dept: Department = Department.Engineering;

// Often simpler: a union of string literals does almost the same job
type DepartmentLite = "Engineering" | "Sales" | "HR";
let dept2: DepartmentLite = "Sales";
```

Coming from C#, `enum` will feel like home. But here's a heads-up: in TypeScript, an `enum` actually generates extra JavaScript code at runtime (remember, most types vanish — enums are a rare exception that don't fully vanish). For that reason, the community often prefers the **string-literal union** (`type DepartmentLite`) — it does nearly the same job, reads just as clearly, and leaves *zero* runtime code behind. Use enums if you like them, but know the union is the lighter, more idiomatic choice in modern React/TS code.

---

7. Typing functions

You annotate each parameter, and you can annotate the return type after the parameter list. Arrow functions work the same way.

```
// typed parameters + return type
function raiseSalary(current: number, percent: number): number {
  return current + current * (percent / 100);
}

// arrow function, same idea
const fullName = (first: string, last: string): string => `${first} ${last}`;

// return type can be inferred — TS knows this returns boolean
const isSenior = (salary: number) => salary > 100000;

// void = returns nothing
const logName = (name: string): void => {
  console.log(name);
};
```

C# analogy: `int RaiseSalary(int current, int percent)` — same idea, the return type just moves to after the parentheses. `void` means "returns nothing," identical to C#. And note `isSenior`: you can skip the return type and let inference figure it out. A good habit is to annotate parameters always, and let the return type infer unless you want to be explicit.

---

8. Generics — reusable typed code

A **generic** is code that works with *any* type, decided by whoever uses it. The `<T>` is a **type parameter** — a placeholder that gets filled in at the call site. This is the exact same concept as C# generics (`List<T>`, `Task<T>`).

```
// generic function: <T> is a placeholder type, decided by the caller
function firstOf<T>(items: T[]): T | undefined {
  return items[0];
}

const n = firstOf([1, 2, 3]);        // T = number  → number | undefined
const s = firstOf(["a", "b"]);       // T = string  → string | undefined
```

Now let's build up to something real that ties back to yesterday's mock API. Remember your Day 1 API returned employees wrapped in a response? We can type that wrapper *once* and reuse it for any payload:

```
// generic wrapper that ties back to Day 1's mock API
interface Employee {
  id: number;
  firstName: string;
  lastName: string;
}

interface ApiResponse<T> {
  data: T;
  success: boolean;
  error?: string;
}

// the API can return ONE employee ...
async function getEmployee(id: number): Promise<ApiResponse<Employee>> {
  return { data: { id, firstName: "Ada", lastName: "Lovelace" }, success: true };
}

//  ... or a LIST of employees, same wrapper, different T
async function getEmployees(): Promise<ApiResponse<Employee[]>> {
  return { data: [], success: true };
}
```

See the payoff: `ApiResponse<Employee>` and `ApiResponse<Employee[]>` reuse the *same* wrapper shape, but the `data` field is correctly typed each time. This is exactly like `Task<Employee>` vs `Task<List<Employee>>` in C#. You'll see this generic-response pattern everywhere in real React apps.

---

9. Utility types — Partial, Pick, Omit, Required, Readonly, Record

Utility types are built-in helpers that take an existing type and hand you back a tweaked version of it. Think of them as ready-made transformations so you don't rewrite shapes by hand. All examples use our `Employee`.

```ts
interface Employee {
  id: number;
  firstName: string;
  lastName: string;
  department: string;
  salary: number;
  isActive: boolean;
}

// Partial<T>: every property becomes optional — great for "update" payloads
type EmployeeUpdate = Partial<Employee>;
const patch: EmployeeUpdate = { salary: 90000 };

// Pick<T, keys>: keep only the listed properties
type EmployeeCard = Pick<Employee, "id" | "firstName" | "lastName">;
const card: EmployeeCard = { id: 1, firstName: "Ada", lastName: "Lovelace" };

// Omit<T, keys>: everything EXCEPT the listed properties
type NewEmployee = Omit<Employee, "id">; // id assigned later by the server
const draft: NewEmployee = {
  firstName: "Grace",
  lastName: "Hopper",
  department: "Engineering",
  salary: 95000,
  isActive: true,
};

// Required<T>: every property becomes required (opposite of Partial)
type FullEmployee = Required<Employee>;

// Readonly<T>: every property becomes read-only (frozen)
type FrozenEmployee = Readonly<Employee>;

// Record<K, V>: an object whose keys are K and values are V (like a dictionary)
type CountByDept = Record<string, number>;
const counts: CountByDept = { Engineering: 12, Sales: 5 };
```

Plain-English summary of each:

- `Partial<Employee>` — makes every field optional. Perfect for an "edit" or "patch" form where you only send the fields that changed.
- `Pick<Employee, ...>` — cherry-pick just the fields you want. Great for a small display card.

- `Omit<Employee, ...>` — everything *except* the fields you name. Classic use: a "create new" shape with no `id` yet.
- `Required<Employee>` — the opposite of Partial: forces every field to be present.
- `Readonly<Employee>` — freezes every field. Like wrapping the whole object in `readonly`.
- `Record<string, number>` — builds a dictionary/map type. This is your C# `Dictionary<string, int>`.

These save you a ton of repetition. Instead of hand-writing a `NewEmployee` interface that duplicates `Employee` minus `id`, you write `Omit<Employee, "id">` and it stays in sync automatically when `Employee` changes.

---

10. Type narrowing — teaching TypeScript which type you have

When a value could be several types (a union), you often need to figure out *which one it actually is* at runtime, then act accordingly. Getting TypeScript to see "in this branch it's definitely a string" is called **narrowing** (narrowing a wide union down to one specific type). There are three everyday tools.

a) `typeof` checks — for primitives:

```
// typeof narrowing: check the runtime type, TS narrows inside the branch
function format(value: string | number): string {
  if (typeof value === "string") {
    return value.toUpperCase(); // here TS knows value is string
  }
  return value.toFixed(2);      // here TS knows value is number
}
```

b) The `in` operator — check whether a property exists on an object:

```
// 'in' narrowing: check whether a property exists on the object
interface Manager {
  name: string;
  reports: number;
}
interface Contractor {
  name: string;
  hourlyRate: number;
}

function describe(person: Manager | Contractor): string {
  if ("reports" in person) {
    return `${person.name} manages ${person.reports} people`;
  }
  return `${person.name} bills $${person.hourlyRate}/hr`;
}
```

c) Discriminated unions — the cleanest pattern. You give each type in the union a shared **tag** field (a literal type like `status`), and then a `switch` on that tag lets TypeScript know *exactly* which type you're in each branch:

```
// discriminated union: a shared "tag" field tells the branches apart
type LoadingState = { status: "loading" };
type SuccessState = { status: "success"; data: string[] };
type ErrorState = { status: "error"; message: string };
type FetchState = LoadingState | SuccessState | ErrorState;

function render(state: FetchState): string {
  switch (state.status) {
    case "loading":
      return "Loading…";
    case "success":
      return `Loaded ${state.data.length} items`; // TS knows .data exists here
    case "error":
      return `Failed: ${state.message}`;           // TS knows .message exists here
  }
}
```

This `loading | success | error` pattern is *the* standard way to model "fetching data" state in React, so it's well worth getting comfortable with now. Notice inside `case "success"` you can safely reach `state.data`, but you couldn't in the other branches — TypeScript narrows it for you based on the

`status` tag. (Bonus: under strict mode, if you forget to handle one case, TS will warn you the function might not return — a free safety net.)

---

11. `any` vs `unknown`

Two "I'm not sure of the type" escape hatches — but they behave very differently.

- `any` turns *off* all type checking for that value. You can do anything to it, and TypeScript won't complain — which means it also won't catch your bugs. It's a hole in your safety net. **Avoid it.**
- `unknown` is the *safe* version: it says "I don't know the type yet, so you must check before you use it." You can't touch it until you narrow it (usually with `typeof` or an `in` check).

```
// any: turns OFF all checking — you lose every safety net. Avoid it.
let loose: any = "hello";
loose.foo.bar(); // compiles, but will crash at runtime — no warning

// unknown: safe "I don't know yet". You MUST check before using it.
let safe: unknown = "hello";

// ❌ Error: 'safe' is of type 'unknown'.
// safe.toUpperCase();

if (typeof safe === "string") {
  console.log(safe.toUpperCase()); // fine — we narrowed it first
}
```

Rule of thumb: reach for `unknown` (safe) when a value's type is genuinely uncertain — like data coming back from an external API. Treat `any` as a last resort you'll want to explain in a code review. There's no C# equivalent to `any`; C#'s `object` is closer to `unknown` (you must cast/check before using it), and C#'s `dynamic` is the closest to `any` — and just as risky.

---

12. tsconfig & strict mode

`tsconfig.json` is the settings file for the TypeScript compiler — it tells `tsc` how to check and build your code. The one setting that matters most for you right now is `"strict": true`.

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "ESNext",
    "strict": true,
    "noEmit": true
  }
}
```

`strict` is actually a bundle switch that flips on several stricter checks at once. The two you'll feel most:

- `strictNullChecks` — `null` and `undefined` are no longer allowed to sneak into every type. If something *can* be null, you must say so (`string | null`) and handle it. This kills a huge class of "cannot read property of undefined" crashes.
- `noImplicitAny` — TypeScript won't silently give up and assign `any` when it can't figure out a type; it forces you to be explicit.

```
// Under strict mode, this parameter can't secretly be undefined:
function greet(name: string): string {
  return `Hi, ${name}`;
}

// ❌ Error under strict: Argument of type 'undefined' is not assignable to 'string'.
// greet(undefined);

// strict also forces you to handle null/undefined explicitly:
function firstChar(text: string | null): string {
  if (text === null) return "";
  return text[0] ?? "";
}
```

Why it's worth it: strict mode is what makes TypeScript actually *catch* your mistakes instead of just decorating your code. If you're used to C#'s nullable-reference-types warnings (`string?`), `strictNullChecks` will feel very familiar — it's the same "don't let null bite you" idea. Turn `strict` on from day one and never look back; retrofitting it onto a big loose codebase later is painful.

One-line summary of the whole day

TypeScript adds C#-style types on top of JavaScript to catch mistakes before you run — you describe shapes with **interfaces/types**, tighten them with **optional/readonly/unions/literals**, reuse them with **generics** and **utility types**, prove which type you have at runtime with **narrowing**, prefer `unknown` **over**

`any`, and let **strict mode** enforce it all — and then every one of those types vanishes at compile time, leaving plain JavaScript to run.

3 Common Mistakes to Avoid

You know C#. TypeScript's type system feels like home in a lot of ways. But there are a few traps that catch people coming from C# and from plain JS. Here they are, with ❌ (the slip-up) and ✅ (the fix). All the ✅ code here compiles under strict mode.

---

1. Overusing `any` — it turns TypeScript OFF

`any` means "stop checking this value." It is like casting everything to `dynamic` in C#. You lose every bit of help.

```
// ❌ any = no safety at all
function parseBad(json: string): any {
  return JSON.parse(json);
}
const total: number = parseBad("{}").salary; // compiles, but could blow up
```

```
// ✅ Return unknown, then check before using
function parseGood(json: string): unknown {
  return JSON.parse(json);
}
```

Rule of thumb: if you are reaching for `any`, reach for `unknown` instead (see #6). Save `any` for the rare case you truly cannot type something.

---

2. Using `as` to silence an error instead of fixing it

`as` is a *type assertion*. That is a fancy way of saying "trust me, compiler, I know what this is." The compiler stops arguing. In C# an `(Employee)obj` cast is checked at runtime and throws if wrong. TypeScript's `as` is **not** checked — it just goes quiet. If you are wrong, you find out with a crash later.

```
// ❌ as hides the problem
const data: unknown = { firstName: "Sam" };
const emp = data as Employee;   // compiler trusts you
emp.salary.toFixed(2);          // 💥 salary is actually undefined
```

```
// ✅ Check it for real with a type guard
function isEmployee(x: unknown): x is Employee {
  return (
    typeof x === "object" &&
    x !== null &&
    "salary" in x &&
    typeof (x as Record<string, unknown>).salary === "number"
  );
}

if (isEmployee(data)) {
  data.salary.toFixed(2); // safe here
}
```

If you *must* use `as`, treat it as a red flag, not a habit.

---

3. TypeScript types are ERASED at runtime — no free validation

This is the big one coming from plain JS *and* from C#. Types exist only while you write and compile. When the code runs, they are **gone** — completely stripped out. There is no reflection, no runtime type object like C# has.

So the type on an API response is just a **promise you made to yourself**, not something TypeScript enforces. The server can send you garbage and TypeScript will never know.

```
// ❌ You TOLD TypeScript it is an Employee, but nobody checked
async function loadEmployeeBad(): Promise<Employee> {
  const body: unknown = await res.json();
  return body as Employee; // pure hope
}
```

```
// ✅ Actually validate the data at the boundary
async function loadEmployeeGood(): Promise<Employee> {
  const body: unknown = await res.json();
  if (!isEmployee(body)) {
    throw new Error("Bad employee data from API");
  }
  return body; // now it really is an Employee
}
```

Remember Day 1's mock API with the deliberate failure? Same idea: you still have to *check* what comes back. (Later, libraries like Zod make this validation short and neat.)

---

4. Assuming enums behave like C# enums

TypeScript enums look like C# enums but have surprises. A plain `enum` is **numeric** by default, and it builds a **reverse map** (number → name) that you did not ask for. That extra behaviour trips people up.

```
// ❌ Surprise: numeric enum with reverse mapping
enum Department { Sales, Engineering }
Department.Sales;      // 0
Department[0];         // "Sales"  ⟵ reverse lookup you didn't expect
```

For most cases, a **string-literal union** is simpler, lighter, and shows real text in your data:

```
// ✅ Often nicer than an enum
type Department = "Sales" | "Engineering" | "HR";
const dept: Department = "Sales"; // just a string, no hidden object
```

Enums are fine when you want a named set with a real runtime object. But when in doubt, a string union is the easy default.

---

5. Not turning on strict mode

Without `strict`, TypeScript is soft and lets a lot slide (like silently allowing `null`). It is like turning off half your compiler warnings in C#. Turn it on and leave it on.

```
// ✅ tsconfig.json
{
  "compilerOptions": {
    "strict": true
  }
}
```

`"strict": true` flips on a whole bundle of good checks at once (including strict null checks). Start every project with it.

---

6. Confusing `unknown` and `any`

They look similar but are opposites in spirit.

- `any` = "I give up, don't check anything." (unsafe)

- `unknown` = "I don't know yet, so **check me before you use me**." (safe)

```
// ❌ any lets you do anything, even wrong things
function handleAny(value: any) {
  return value.toUpperCase(); // compiles, crashes if value is a number
}
```

```
// ✅ unknown forces you to narrow first
function handleUnknown(value: unknown) {
  if (typeof value === "string") {
    return value.toUpperCase(); // safe
  }
  return "";
}
```

Prefer `unknown` at the edges of your app (API data, JSON, user input). It is the safe version of `any`.

---

7. Ignoring "possibly undefined / null" errors

Under strict mode, TypeScript warns when a value might be `undefined` or `null`. This is basically C# nullable reference types (`Employee?`) — a helper, not a nag. Don't silence it with `!` or `as` ; **handle it**.

Watch out: array `.find()` returns the item **or** `undefined` if nothing matches (just like plain JS).

```
// ✅ Handle the undefined case
function firstNameOf(list: Employee[]): string {
  const first = list.find((e) => e.isActive);
  if (first === undefined) {
    return "none";
  }
  return first.firstName; // safe: we ruled out undefined
}
```

Avoid the non-null assertion `first!.firstName` — that just tells the compiler to look away, same problem as `as`.

---

8. Over-annotating when inference would do

TypeScript is good at guessing types. You do not need to spell out every one. Coming from C#'s explicit style, it is tempting to annotate everything — but let inference carry the obvious cases and save the typing.

```
// ❌ Noisy — the types are obvious
const count: number = 5;
const names: string[] = ["a", "b"].map((n: string): string => n.toUpperCase());
```

```
// ✅ Let TypeScript infer the obvious stuff
const count = 5;                                    // inferred: number
const names = ["a", "b"].map((n) => n.toUpperCase()); // inferred: string[]
```

Where you **should** still annotate: function parameters, function return types on public functions, and empty containers (like `const list: Employee[] = []`). Those are the spots where inference can't help.

---

9. Mixing up `interface` vs `type` for no reason

Both can describe an object shape, and for that job they are nearly interchangeable. People waste time agonising over which to use. Don't.

```
// ✅ Both are fine for an object shape
interface EmployeeI { id: number; name: string; }
type EmployeeT = { id: number; name: string; };
```

Simple guideline:

- Use `interface` for object shapes (like a C# `interface` or a data class). It is the common choice for models and props.
- Use `type` when you need unions, string-literal unions, or other things interfaces can't do — e.g. `type Department = "Sales" | "HR"`.

Pick one, stay consistent, and move on. This is not worth debating.

Quick recap: don't switch TypeScript off (`any` , missing strict), don't lie to it (`as` , `!`), remember types vanish at runtime so **validate API data**, prefer string unions over enums when unsure, use `unknown` at the edges, handle possibly-undefined, and let inference do the easy work.

4 Concept Notes — Recall & Interview (copy by pen)

> This section is your TypeScript concept-recall sheet — copy it by pen to lock the ideas in and to answer interview questions fast. Everything is point-to-point, with a one-line C#/.NET comparison wherever it helps.

Foundations — What & Why

What TypeScript is & why

Definition:

- TypeScript is JavaScript + a type layer that checks your code at compile time, then compiles down to plain JS.
- The types are erased at runtime — the browser only ever runs JavaScript, it never sees your types.

✅ **Advantages:**

- Catches bugs (typos, wrong shapes, missing fields) before you run the code.
- Great editor help: autocomplete, refactor-safe renames, inline docs.
- Self-documenting — the types tell you what a function expects.

❌ **Disadvantages / watch-outs:**

- Extra build step (`tsc`) and some upfront type-writing.
- Types are gone at runtime — you cannot check "is this really an Employee?" at runtime from a type alone.

💡 **Interview tip:**

- "Types are compile-time only and fully erased at runtime" is the single most common TS interview point — say it exactly.

📝 **Example:**

- `let name: string = "Het";` — compiles to `let name = "Het";` (type stripped out).

Type annotation vs type inference

Definition:

- Annotation = you write the type yourself (`let x: number = 5`).
- Inference = TypeScript guesses the type from the value (`let x = 5` → `x` is `number`).

✅ **Advantages:**

- Inference means less typing while staying type-safe.
- Annotations make function inputs/returns explicit and clear.

❌ **Disadvantages / watch-outs:**

- Over-annotating everything is noise — trust inference for simple locals.
- Always annotate function parameters (they are not inferred).

📝 **Example:**

- `const dept = "Sales";` — inferred as `string`, no annotation needed.

Primitive types

Definition:

- The basic building-block types you put after a colon: `string`, `number`, `boolean`.
- JS primitives also include `null`, `undefined`, `bigint`, `symbol`; TypeScript adds two type-only ones: `void` and `never`.

✅ **Advantages:**

- Same numeric type for everything — no `int` / `long` / `double` split like C#.

❌ **Disadvantages / watch-outs:**

- `null` and `undefined` are two different things in JS/TS (C# has just `null`).
- `void` / `never` are TS type additions — they are not real JS runtime values.

📝 **Example:**

- `let salary: number = 50000;` — one `number` type covers ints and decimals.

Shaping Data — Interfaces, Types, Modifiers

interface vs type alias

Definition:

- `interface` names an object shape (a contract for what an object must contain).
- `type` alias names ANY type — object shapes, but also unions, intersections, and primitives.

✅ **Advantages:**

- `interface` can be extended/merged and reads well for object contracts.
- `type` is more flexible — only `type` can name a union like `"A" | "B"`.

❌ **Disadvantages / watch-outs:**

- They overlap a lot for objects — don't overthink it.
- Rule of thumb: `interface` for object shapes others may extend, `type` for unions or anything non-object.

💡 **Interview tip:**

- Classic question: "interface vs type?" — key answer: only `type` can express unions; interfaces support declaration merging.

📝 **Example:**

- `interface Employee { id: number }` vs `type Status = "Active" | "Resigned"` — interface = shape, type = union.

Optional (?) & readonly

Definition:

- `?` marks a property that might not be there (`middleName?: string`).
- `readonly` means the property can't be reassigned after the object is created.

✅ **Advantages:**

- `?` forces you to check for `undefined` , preventing null-style bugs.
- `readonly` documents "set once, never change" intent.

❌ **Disadvantages / watch-outs:**

- `readonly` is compile-time only — it does NOT freeze the object at runtime.
- An optional property's type is really `T | undefined` — handle the undefined case.

📝 **Example:**

- `interface E { readonly id: number; name?: string }` — `id` can't change, `name` may be missing.

Unions, Literals & Enums

Union types & literal types

Definition:

- A union means "one of these types" using `|` (`string | number`).
- A literal type is one exact value (`"Active"`), and a literal union restricts to a fixed set of exact values.

✅ **Advantages:**

- Literal unions give you a lightweight, type-safe fixed set of options.
- No runtime code — pure compile-time checking.

❌ **Disadvantages / watch-outs:**

- C# has no direct equivalent to string unions — this is a genuinely new tool for you.
- Before using union-specific properties you must narrow (check which type it is).

📝 **Example:**

- `type Dept = "Engineering" | "Sales" | "HR";` — only those three strings allowed.

Enums vs string-literal union

Definition:

- An `enum` is a named set of constants (`enum Status { Active, OnLeave }`) — and it creates real code at runtime.

- A string-literal union (`type Status = "Active" | "OnLeave"`) does the same job with zero runtime code.

✅ **Advantages:**

- Enum: familiar C# feel, auto-numbering from 0, one named group.
- String union: simpler, lighter, values match your API strings exactly.

❌ **Disadvantages / watch-outs:**

- Enums emit JS at runtime (one of the few TS features that do) — extra bundle weight.
- Many teams prefer string unions — know both, you'll see both.

💡 **Interview tip:**

- "enum vs union?" — enums are runtime objects; unions are erased. Prefer unions unless you need a runtime value.

📝 **Example:**

- C# comparison: `enum Status { Active }` is basically C#'s `enum` — same auto-numbering from 0.

Generics & Utility Types

Generics

Definition:

- Generics let one type or function work with any type you plug in later, while staying type-safe (`<T>` is the placeholder).
- Perfect for a reusable API wrapper: same shape, different data inside.

✅ **Advantages:**

- Write once, reuse for every entity with full type checking.
- No loss of type info (unlike using `any`).

❌ **Disadvantages / watch-outs:**

- Over-generic code gets hard to read — only add `<T>` when something truly varies.

💡 **Interview tip:**

- Be ready to write `ApiResponse<T>` on the spot — a very common TS interview ask.

📝 **Example:**

- `interface ApiResponse<T> { success: boolean; data: T }` — reuse as `ApiResponse<Employee>` or `ApiResponse<Employee[]>` .
- C# comparison: identical in spirit to C# generics — `ApiResponse<T>` is just like `List<T>` .

Utility types (Partial / Pick / Omit / Required / Readonly / Record)

Definition:

- Built-in helpers that build a NEW type from an existing one, so you don't repeat field lists.
- `Partial` all-optional, `Required` all-mandatory, `Pick` keep some, `Omit` drop some, `Readonly` lock all, `Record` build key→value map.

✅ **Advantages:**

- Derive create/update/summary/read-only versions of `Employee` without copy-pasting fields.
- Stay in sync — change the base type and all derived types update.

❌ **Disadvantages / watch-outs:**

- Chaining too many together gets unreadable — name the intermediate types.

💡 **Interview tip:**

- Know each one's one-liner: `Partial` = update payload, `Omit<Employee,"id">` = new employee with no id yet.

📝 **Example:**

- `type CountByDept = Record<Dept, number>;` — object keyed by each department to a number.
- C# comparison: `Record<K, V>` ~ `Dictionary<K, V>`; C# has no auto Partial/Pick — you'd hand-write separate DTOs.

Narrowing & Safety

Type narrowing (typeof / in / discriminated unions)

Definition:

- Narrowing = TypeScript figures out a more specific type inside an `if` block, based on a check you wrote.
- Common checks: `typeof x === "string"`, `"key" in obj`, or a shared literal tag on a discriminated union.

✅ **Advantages:**

- After a check, TS lets you safely use the matching type's properties.
- Discriminated unions make handling "one of several shapes" bulletproof.

❌ **Disadvantages / watch-outs:**

- `typeof` only distinguishes JS primitives, not your interfaces.
- Forgetting to narrow a union = compile error when you touch a union-only property.

💡 **Interview tip:**

- "How do you narrow a union?" — `typeof` for primitives, `in` for object keys, a discriminant (`kind`) field for object unions.

📝 **Example:**

- `if (typeof x === "number") { x.toFixed() }` — inside the `if`, `x` is a `number`.

any vs unknown

Definition:

- `any` turns OFF all type checking — anything goes, no safety.
- `unknown` is the safe version — you must narrow/check it before you can use it.

✅ **Advantages:**

- `unknown` keeps you honest — forces a check before use.
- `any` is an occasional escape hatch for messy migrations.

❌ **Disadvantages / watch-outs:**

- `any` spreads and silently disables safety — avoid it.
- `unknown` needs narrowing before every use (that's the point).

💡 **Interview tip:**

- "any vs unknown?" — both hold anything, but `unknown` won't let you touch it until you narrow; prefer `unknown`.

📝 **Example:**

- `let a: any` skips checks; `let u: unknown` must be checked first.
- C# comparison: `any` ~ `dynamic` (no checking); `unknown` ~ `object` (safe, must cast/check).

Structural typing (vs C# nominal typing)

Definition:

- TypeScript matches types by SHAPE, not by name — if an object has the right fields, it fits.
- C# is nominal — a type matches only if it explicitly declares that name/interface.

✅ **Advantages:**

- Very flexible — any object with the right shape "just works," no explicit `implements` needed.

❌ **Disadvantages / watch-outs:**

- Two unrelated types with the same shape are interchangeable — sometimes surprising.

💡 **Interview tip:**

- "Structural vs nominal typing?" — TS = duck typing by shape; C# = matches by declared name.

📝 **Example:**

- `{ id: 1, name: "x" }` fits any type needing `{ id: number; name: string }` — no name match required.

Type erasure at runtime

Definition:

- All types, interfaces, and type aliases are deleted during compile — none exist when the code runs.

❌ **Disadvantages / watch-outs:**

- You can't do `if (x instanceof MyInterface)` — interfaces don't exist at runtime.
- To check shape at runtime, inspect actual values (`"id" in x`) or use a type guard.

💡 **Interview tip:**

- "Can you check a TS type at runtime?" — no, types are erased; you must check the real value.

📝 **Example:**

- `interface E {}` compiles to nothing — zero runtime footprint.

tsconfig strict mode

Definition:

- `tsconfig.json` is the compiler's settings file; `"strict": true` turns on all the strong safety checks at once.
- Includes `strictNullChecks` (null/undefined must be handled) and `noImplicitAny` (no accidental `any`).

✅ **Advantages:**

- Catches the most bugs — null safety plus no silent `any` .
- Industry-standard baseline for new projects.

❌ **Disadvantages / watch-outs:**

- Turning it on for old code surfaces many errors at once — expect cleanup.

📝 **Example:**

- `{ "compilerOptions": { "strict": true } }` — one flag enables the full safety set.
- C# comparison: think of `tsconfig.json` as your `.csproj` / build settings for TypeScript.

High-Value Extras

Type assertion (as) & non-null assertion (!)

Definition:

- `as` tells the compiler "trust me, treat this value as this type" (`value as string`).
- `!` (non-null assertion) tells the compiler "this is definitely not null/undefined" (`user!.name`).

✅ **Advantages:**

- Handy when YOU know more than the compiler (e.g. DOM lookups, parsed JSON).

❌ **Disadvantages / watch-outs:**

- Neither is checked at runtime — if you're wrong, you get a runtime crash, not a compile error.
- Overusing `!` hides real null bugs — prefer a proper check.

💡 **Interview tip:**

- "Does `as` convert or validate?" — no, it only silences the compiler; there is no runtime check.

📝 **Example:**

- `const el = document.getElementById("x") as HTMLInputElement;` — assert the element type.
- C# comparison: `as` ~ a cast, but NOT runtime-checked (unlike C#'s `as`, which returns null on failure).

Intersection types (&) vs union (|)

Definition:

- Intersection `A & B` = must satisfy BOTH (combine all properties).
- Union `A | B` = must satisfy EITHER one.

✅ **Advantages:**

- `&` is great for merging shapes (base fields + extra fields).
- `|` is great for "one of several allowed types."

❌ **Disadvantages / watch-outs:**

- Easy to mix up: `&` = AND (more required), `|` = OR (fewer guaranteed).
- Intersecting conflicting primitive types can produce `never`.

💡 **Interview tip:**

- "& vs |?" — `&` combines (all members), `|` chooses (one member); remember AND vs OR.

📝 **Example:**

- `type Staff = Employee & { managerId: number };` — has all Employee fields plus `managerId`.

keyof and typeof (type operators)

Definition:

- `keyof T` produces a union of T's property names (`keyof Employee` → `"id" | "name" | ...`).
- `typeof value` (in type position) grabs the TYPE of an existing value/object.

✅ **Advantages:**

- Together they keep types in sync with real objects — no hand-copying keys.
- Powers type-safe property access helpers.

❌ **Disadvantages / watch-outs:**

- `typeof` here is the TYPE operator — different from the runtime `typeof` used in narrowing.

📝 **Example:**

- `type Keys = keyof Employee;` gives the field-name union; `type T = typeof emp;` gives `emp`'s type.

User-defined type guards (x is Type)

Definition:

- A function whose return type is `x is Type` — when it returns true, TS narrows `x` to that type.

✅ **Advantages:**

- Lets you reuse a custom shape-check everywhere and get narrowing for free.
- Perfect for validating unknown/API data into a real type.

❌ **Disadvantages / watch-outs:**

- YOU are responsible for the logic being correct — TS trusts your `is` claim.

💡 **Interview tip:**

- "How do you narrow a custom type?" — write a guard returning `x is Employee`.

📝 **Example:**

- `function isEmployee(x: unknown): x is Employee { return typeof x === "object" && x !== null && "id" in x; }` — narrows after the check.

as const

Definition:

- `as const` freezes a value to its most specific literal type and makes it deeply `readonly`.

✅ **Advantages:**

- Turns `"Active"` from `string` into the exact literal `"Active"`.
- Great for fixed config objects and building literal unions from arrays.

❌ **Disadvantages / watch-outs:**

- Everything becomes `readonly` — you can't mutate it afterward.

📝 **Example:**

- `const roles = ["admin", "user"] as const;` — type is the tuple `readonly ["admin", "user"]`, not `string[]`.

Generic constraints (extends)

Definition:

- `extends` limits what a generic `T` can be, so you can safely use certain properties (`<T extends { id: number }>`).

✅ **Advantages:**

- Keeps generics flexible but guarantees the members you rely on exist.

❌ **Disadvantages / watch-outs:**

- Note `extends` here means "must be assignable to" — it's a constraint, not class inheritance.

💡 **Interview tip:**

- "How do you require a generic to have a property?" — constrain it with `extends`.

📝 **Example:**

- `function byId<T extends { id: number }>(x: T) { return x.id; }` — T must have an `id`.
- C# comparison: like a C# generic constraint `where T : ISomething`.

never vs void

Definition:

- `void` = a function returns nothing (it finishes, just no value).
- `never` = a function NEVER returns at all (it always throws or loops forever).

✅ **Advantages:**

- `never` powers exhaustiveness checks — catch a missed case in a switch at compile time.

❌ **Disadvantages / watch-outs:**

- Don't confuse them: `void` finishes normally with no value; `never` can't finish.
- A value can be `void` (undefined); a value can never be `never`.

💡 **Interview tip:**

- "never vs void?" — `void` = returns nothing; `never` = returns nothing because it can't return.

📝 **Example:**

- `function log(): void {}` vs `function fail(): never { throw new Error(); }` — no value vs never returns.

Index signatures ([key: string]: T)

Definition:

- An index signature says "any key of this type maps to this value type" — for objects with unknown/dynamic keys.

✅ **Advantages:**

- Type dictionary-like objects where you don't know the keys ahead of time.

❌ **Disadvantages / watch-outs:**

- Loosens key safety — any string key is allowed, so typos aren't caught.
- Often `Record<K, V>` is cleaner when the keys are a known set.

📝 **Example:**

- `interface Scores { [name: string]: number }` — any string key maps to a number.
- C# comparison: like a `Dictionary<string, int>` — arbitrary string keys, fixed value type.